What is Data?

Data = raw facts, observations, measurements or records that can be stored and processed.

Data can include:

Numbers + Text + Images + Audio + Video + Logs + Sensor readings + Location information + Transactions

Where is so much data coming from?

Sources of Data

Explain that modern data is generated from many sources:

- Smartphones
- Social media
- Websites
- Online shopping
- Banking transactions
- GPS
- IoT devices
- CCTV cameras
- Sensors
- Hospitals
- Cloud applications

EX

Suppose you order biryani.

Now imagine millions of orders, searches, payments and location updates. Traditional approaches may struggle when data becomes extremely large, fast and complex.

What is Big Data?

Technical definition

Big Data refers to extremely large, rapidly generated, and diverse datasets that can be difficult to store, process and analyze efficiently using traditional data-processing systems.

Important clarification

“If my laptop has 32 GB, then data greater than 32 GB is Big Data for me.”

“There is no universal rule saying 1 GB, 100 GB, or 1 TB automatically becomes Big Data.”

What is “big” depends on the **data, workload, speed, complexity and available infrastructure**.

Big Data is not defined by one fixed file size.

Data Sizes

KB → Kilobyte

MB → Megabyte

GB → Gigabyte

TB → Terabyte

PB → Petabyte

EB → Exabyte

ZB → Zettabyte

YB → Yottabyte

Traditional Data vs Big Data

Traditional Data	**Big Data**

Usually manageable size	Can be massive
Often structured	Structured + semi-structured + unstructured
Often handled by centralized systems	Often benefits from distributed systems
Traditional databases may be sufficient	Distributed storage/processing may be needed

- Structure
- Semi-structure
- Unstructured

5 Vs

“So we know Big Data isn't simply a very large Excel file. Big Data introduces challenges related to amount, speed, different formats, reliability and usefulness.”

1. Volume
2. Velocity
3. Variety
4. Veracity
5. Value

“These 5 Vs help us understand the characteristics and challenges associated with Big Data.”

5 Vs of Big Data

How do we identify whether data has Big Data characteristics?

We commonly explain the characteristics of Big Data using the **5 Vs**.

- Volume refers to the amount or quantity of data being generated and stored.

  VOLUME = AMOUNT OF DATA

- Volume is concerned with **how much**. Velocity is concerned with **how fast data is generated, transmitted, and**

needs to be processed.

VELOCITY = SPEED OF DATA

When we are travelling with google maps,

Location updates, traffic conditions, road events and other signals may need to be processed quickly.

- Variety refers to the different forms, formats and types of data.

  VARIETY = DIFFERENT TYPES OF DATA

- Having a huge amount of data doesn't automatically mean we have good data.

  VERACITY = QUALITY + TRUSTWORTHINESS

Customer 1 → Age 25

Customer 2 → Age 31

Customer 3 → Age -5

Customer 4 → Age 250

Customer 5 → Missing

Can I directly trust this data?

Suppose you're performing sentiment analysis on social-media posts.

Some accounts may be bots, spam accounts, or contain misleading information.

If the underlying data isn't trustworthy, the resulting analysis may also be unreliable.

- A company may have petabytes of data. But if it cannot extract useful information or make better decisions from that data, simply having the data provides limited benefit.

VALUE = USEFUL BUSINESS OUTCOME

Our goal is not just to collect Big Data. Our goal is to extract value from the data.

Hadoop

“Suppose I have a huge dataset, and one machine cannot store or process it efficiently. What can I do?”

Instead of depending only on one powerful computer, what if I use multiple computers together?

“This basic idea is called **distributed computing/storage**. We distribute data and computation across multiple machines.”

Questions

Where is each piece of data?

What if one machine fails?

How do we process everything together?

How do we manage all these machines?

These questions create the need for technologies such as **Hadoop**.

Hadoop

Suppose I have several terabytes or petabytes of data. Can I always depend on one normal laptop to store and process everything efficiently?

What if instead of depending on one computer, I connect many computers and make them work together?

This leads us to the idea of a **distributed system**.

One important technology that became widely used for distributed storage and processing of large datasets is **Apache Hadoop**

1000 Papers

↓

↓ ↓ ↓

Teacher 1 Teacher 2 Teacher 3

300 350 350

↓ ↓ ↓

Results

Apache Hadoop is an open-source framework designed for distributed storage and processing of large datasets across clusters of computers.

Instead of:

BIG DATA

↓

ONE HUGE MACHINE

Hadoop approach:

BIG DATA

↓

↓ ↓ ↓ ↓

Node1 Node2 Node3 Node4

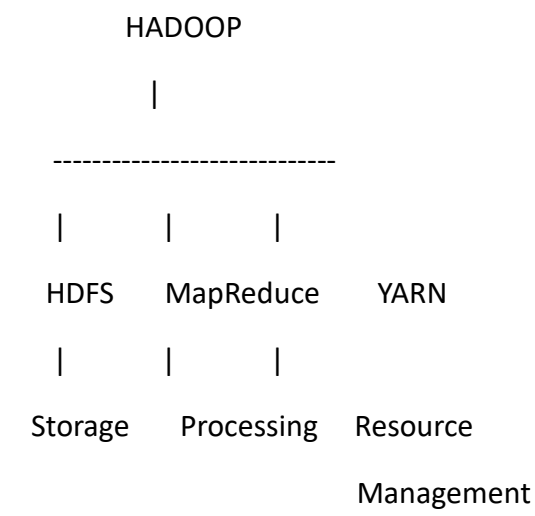
NODE = ONE MACHINE

CLUSTER = GROUP OF CONNECTED MACHINES

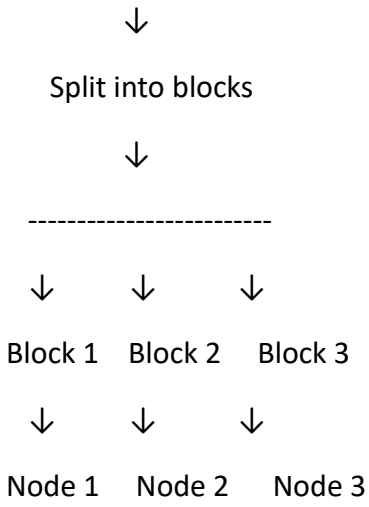
Apache Hadoop :

Apache Hadoop is an Open-Source framework designed for distributed storage and processing of large datasets across clusters of computers.

Hadoop distributes storage and processing across a cluster.



Large File



Amazon Warehouse Analogy

Warehouse → HDFS

Workers → MapReduce processing

Manager → YARN

Different areas → Nodes

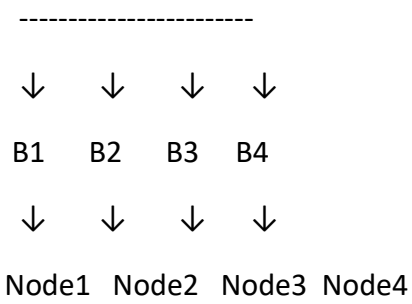
Entire facility → Cluster

HDFS

HDFS = Hadoop Distributed File System

HDFS is a distributed file system designed to store large files across multiple machines in a Hadoop cluster

File = Block



400 MB File

Block 1 → 128 MB

Block 2 → 128 MB

Block 3 → 128 MB

Block 4 → 16 MB

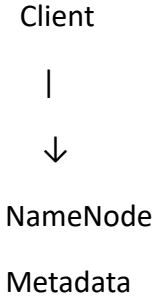
HDFS follows a master-worker architecture

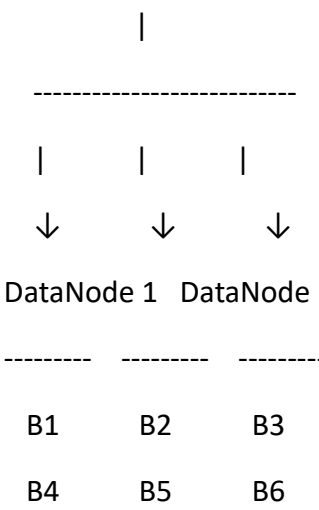
NameNode

Management/metadata role.

DataNodes

Storage/worker role.





The NameNode primarily maintains **metadata about the filesystem**.

EX:NameNode=Librarian

DataNodes are the HDFS worker/storage nodes that store the actual data blocks.

NameNode knows. DataNode stores.

HDFS Reliability & Fault Tolerance

Suppose Block B2 is stored on DataNode 2, and DataNode 2 suddenly crashes. What happens to B2?

Fault tolerance means the system can continue functioning even when some components fail.

HDFS follows an idea at the **block level**

Replication = Maintaining multiple copies of a data block

B1 → DN1, DN2, DN3

B2 → DN2, DN3, DN4

B3 → DN1, DN3, DN4

Replication factor specifies how many copies/replicas of each block HDFS should maintain.

Okay, NameNode knows that B1 has replicas on DN1, DN2 and DN3. But how does the NameNode know whether DN1 is still alive?

EX:Doctor

DataNodes periodically communicate with the NameNode using **heartbeats**.

A heartbeat basically tells the NameNode: **I am alive and functioning.**

Heartbeat tells NameNode that DataNode is alive. But how does NameNode know what blocks that DataNode has?

Block Report

A DataNode periodically sends information about the HDFS blocks it stores to the NameNode.

Heartbeat : Are you alive?

Block Report : What blocks do you have?

Rack Awareness

Data Center

Rack 1	**Rack 2**
DN1	DN4
DN2	DN5
DN3	DN6

Rack awareness means Hadoop/HDFS understands the network topology of the cluster and can use rack information when placing replicas.

Rack 1

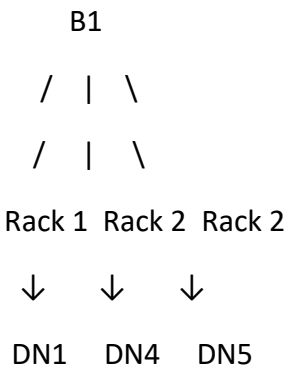
DN1 → B1

DN2 → B1

DN3 → B1

What if that rack experiences a rack-level/network/power failure?

That's why:



But what happens if the NameNode itself fails?”

We will be having 2 concepts

FsImage

Think of it as a persisted **checkpoint/snapshot of the filesystem namespace metadata**.

FsImage

File A

File B

File C

Create File D

Delete File B

Rename File C

Edit Log

Records filesystem namespace changes that occur after a checkpoint.

Checkpointing = FsImage + Edit Log

The traditional Secondary NameNode's primary job is **checkpointing**.

Secondary NameNode ≠ Backup NameNode

Secondary NameNode

"If the NameNode stores all the metadata, what happens if it keeps receiving updates every second?"

- It becomes slow.
- Metadata keeps increasing.

"Exactly. Every time a file is created, deleted, renamed, or modified, the NameNode records these changes. Over time, these records become very large."

The NameNode stores metadata in two important files:

- **FsImage** – A snapshot of the current filesystem metadata.

- **Edit Log** – Records every new change made after the last snapshot.
- The **Secondary NameNode periodically combines the FsImage and Edit Log to create a new updated checkpoint**.

FsImage

+

Edit Log

↓

Secondary NameNode

↓

New Checkpoint (Updated FsImage)

The Secondary NameNode is a helper node that periodically creates checkpoints by merging the FsImage and Edit Log, helping the NameNode manage metadata efficiently.

Modern Hadoop deployments can use **HDFS High Availability**

Active NameNode

Currently handles NameNode operations.

Standby NameNode

Maintains synchronized state so it can take over when failover occurs.

“If you hear **Standby NameNode**, think high availability/failover. If you hear traditional **Secondary NameNode**, think checkpointing.”

The word ‘Secondary’ is misleading. Don't assume that if the NameNode fails, the Secondary NameNode instantly takes over as a live backup.

MapReduce: Distributed Data Processing

We know how to **store** Big Data. But storing data alone doesn't give us any useful result. We need to **process** it.

Suppose I give one student 1,000 exam papers and ask them to count how many students passed. Will it take time?

1000 Papers

↓

↓ ↓ ↓ ↓

250 250 250 250

↓ ↓ ↓ ↓

S1 S2 S3 S4

Parallel Processing = Breaking a large task into smaller tasks and processing multiple tasks simultaneously.

MapReduce is a distributed programming model used to process large datasets in parallel across a cluster.

INPUT DATA

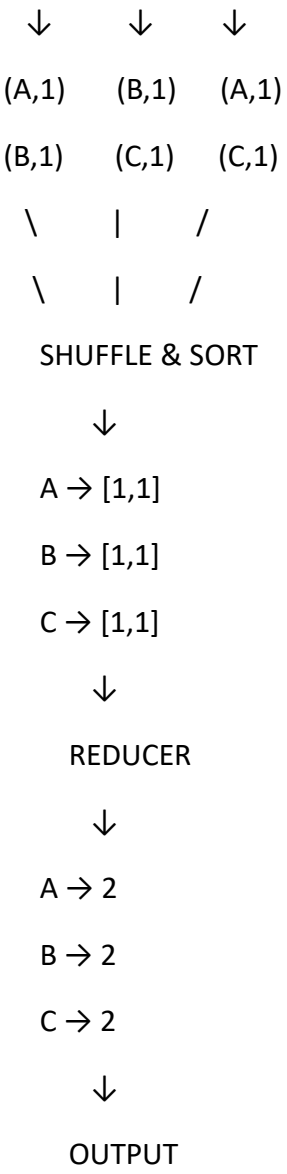
↓

Input Splits

↓

↓ ↓ ↓

Mapper Mapper Mapper

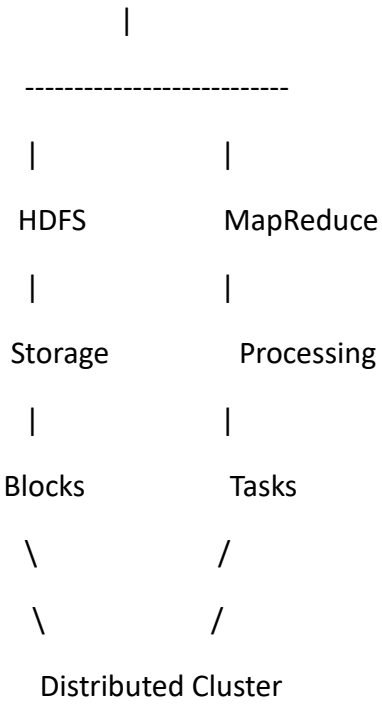


Map processes. Shuffle groups. Reduce aggregates.

Where is our Big Data stored? - **HDFS**

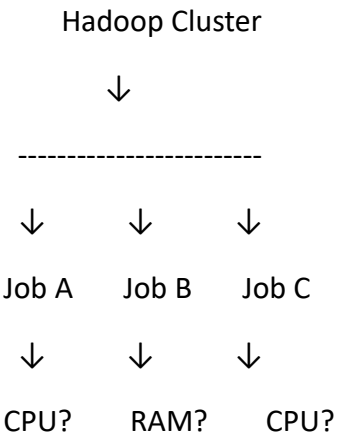
Where are HDFS blocks stored? - **DataNodes.**

HADOOP



YARN: Resource Management in Hadoop

We have 100 machines. Multiple MapReduce jobs are running. One job needs 4 GB memory, another needs 8 GB, another needs CPU resources.



Who decides which application gets which cluster resources?

Who manages resources across all these machines?

YARN = Yet Another Resource Negotiator

HDFS knows **where our data is**. MapReduce describes **how we can process it**. But we still need something to manage the cluster's computational resources. That's where **YARN** comes in.

YARN is Hadoop's resource-management and application-scheduling framework.

YARN doesn't store our actual HDFS files. HDFS handles storage. YARN mainly deals with resources needed to run applications.

Hadoop Cluster

Machine 1 Machine 2 Machine 3

CPU: 8 CPU: 16 CPU: 8

RAM: 16GB RAM: 32GB RAM: 16GB

Multiple applications arrive:

Application A
Application B
Application C
Application D

Where can this application run?"

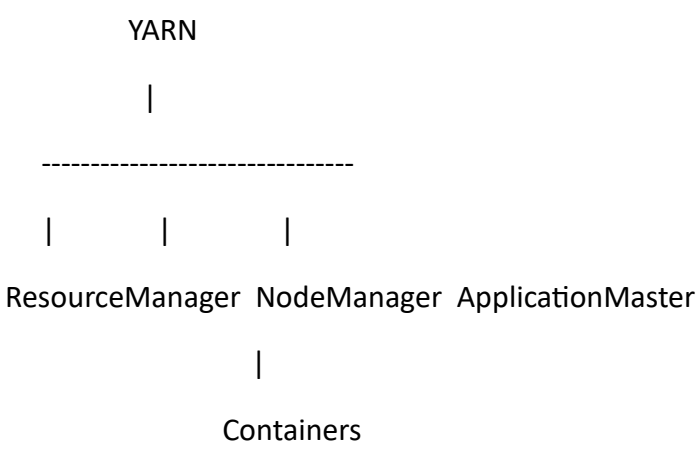
"Which machine has available resources?"

"How much memory should be allocated?"

“How do we track the application's execution?”

YARN provides the framework for this resource management.

Main components of YARN



RM manages the cluster. NM manages a machine. AM manages an application. Container provides resources to run work.

ResourceManager

It deals with:

- Cluster resource allocation
- Scheduling applications/work
- Coordinating resource requests

Think of ResourceManager as the top-level resource authority in YARN.

ResourceManager = College Principal / Central Manager

NodeManager (NM)

NodeManager is the per-node YARN agent that manages containers and reports node/resource information to the ResourceManager.

One ResourceManager manages the cluster; NodeManagers manage individual worker nodes.

ApplicationMaster

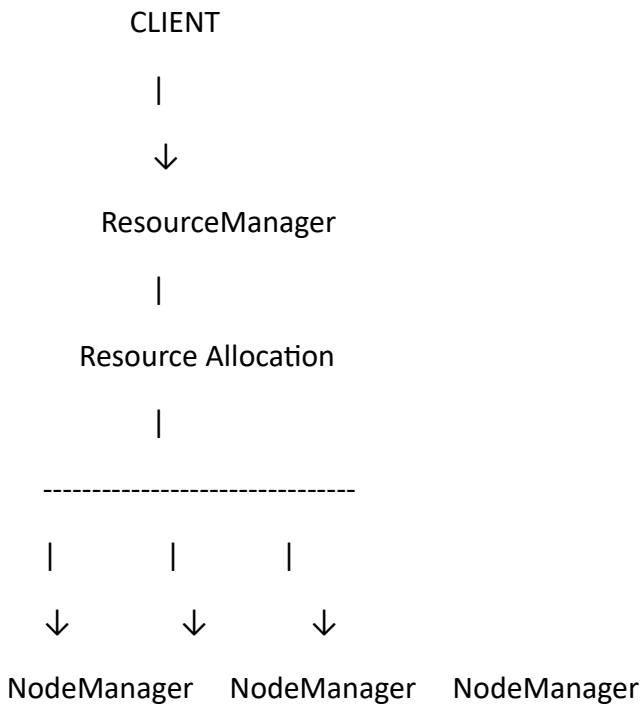
ApplicationMaster manages the lifecycle and resource needs of one application running on YARN.

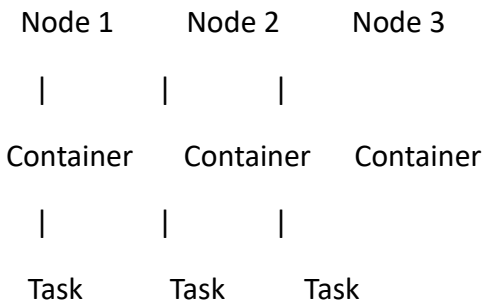
ResourceManager manages resources across the cluster. ApplicationMaster focuses on one application.

Container

ApplicationMaster needs resources to execute tasks. What exactly does YARN allocate?

A YARN container represents an allocation of resources on a node in which application work can execute.





ApplicationMaster

coordinates its

application

Work Flow

1. Client submits application

↓

2. ResourceManager receives it

↓

3. ApplicationMaster starts

↓

4. AM requests resources

↓

5. RM allocates resources

↓

6. NodeManagers launch containers

↓

7. Tasks execute

↓

8. ApplicationMaster monitors

↓

9. Application completes

NameNode = Warehouse inventory/catalog manager

Knows how stored data is organized and where blocks are located.

DataNodes = Warehouses/storage locations

Hold the actual data blocks.

ResourceManager = Central resource manager

Manages compute resources across the cluster.

NodeManager = Local branch manager

Manages containers/execution on one machine.

ApplicationMaster = Project manager

Coordinates one application.

Containers = Allocated workspaces/resources

Where application work runs.

Apache Spark

What if a processing engine could keep reusable intermediate data in memory when beneficial and support many types of analytics through one framework?

That's where **Apache Spark** becomes important.

HADOOP ECOSYSTEM / CLUSTER

HDFS

↓

Distributed Storage

YARN

↓

Resource Management

MapReduce

↓

Distributed Batch Processing

But...

Can distributed processing be

faster and more flexible for

many modern workloads?

↓

APACHE SPARK

We already have MapReduce for Big Data processing. Then why do we need another technology like Spark?

Recall the MapReduce flow:

HDFS

↓

Read Data

↓

Map

↓

Shuffle

↓

Reduce

↓

Write Result

For a single batch job, this model works well.

But now imagine an iterative algorithm.

Read

↓

Process

↓

Write

↓

Read again

↓

Process

↓

Write

↓

Read again...

Repeatedly reading and writing intermediate results can create significant disk I/O. This becomes especially expensive for iterative algorithms and interactive analytics.

Apache Spark is an open-source, distributed computing engine designed for large-scale data processing.

Open-source → publicly developed software.

Distributed → works across multiple machines.

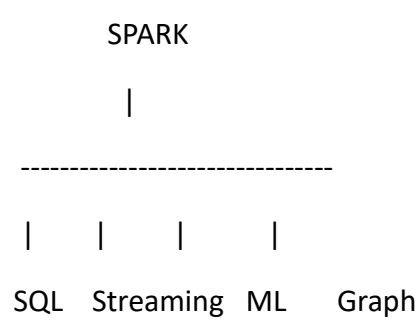
Processing engine → processes data; it is not primarily a distributed storage system like HDFS.

Large-scale → designed to work with large datasets.

Spark is designed to make effective use of memory and can cache reusable data in memory.

Spark's caching is somewhat like keeping frequently reused information close to you rather than repeatedly fetching it from slower storage.

Batch processing



Spark SQL

Structured/semi-structured data processing using DataFrames and SQL.

Structured Streaming

Processing continuously arriving data using Spark's structured APIs.

MLlib

Machine-learning library.

Graph processing

Graph analytics through Spark-related graph APIs/libraries.

Languages supported by Spark

Spark APIs are available in languages including:

Scala
Python
Java
R
SQL

When we use Spark through its Python API, we call it **PySpark**.

Spark Architecture

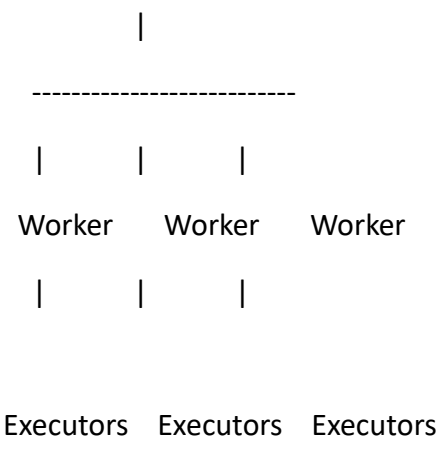
SPARK APPLICATION

|

Driver

|

Cluster Manager



Driver

|

|-- Runs main application logic

|

|-- Builds execution plans

|

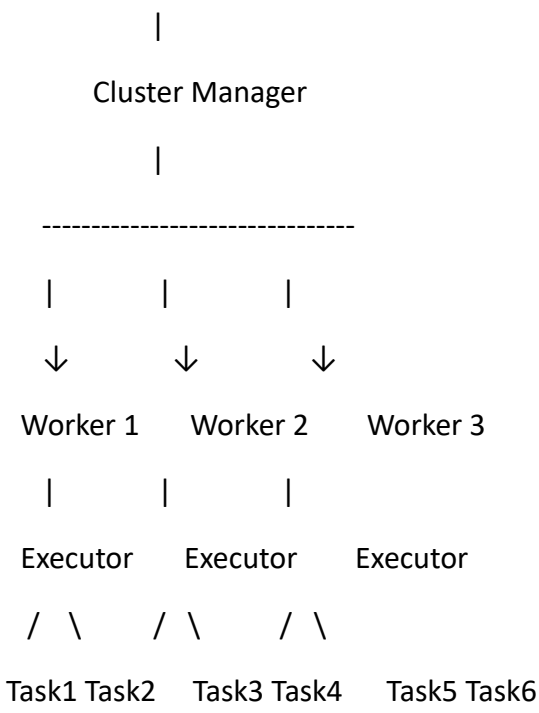
|-- Coordinates/schedules work

|

└-- Communicates with executors

Driver coordinates. Executors execute.

Complete simple architecture



Job

A complete computation triggered by an action.

Stage

A set of tasks that can be executed together without requiring another shuffle boundary.

Task

Work on an individual partition within a stage.

EX: Organize College Event = Job

Registration Stage

Food Stage

Technical Stage

Individual work inside each = Tasks

Spark execution

User Code

↓

Driver

↓

Build execution plan

↓

Job

↓

Stages

↓

Tasks

↓

Executors

↓

Parallel Processing

↓

Result

RDD

RDD = Resilient Distributed Dataset

Resilient

What happens if part of our distributed computation/data is lost because a worker fails?”

Spark can often reconstruct lost RDD partitions using the **lineage** of transformations used to create them.

So:

Resilient means designed to recover lost partitions through lineage/recomputation where possible.

Distributed

Dataset

↓

↓ ↓ ↓

P1 P2 P3

↓ ↓ ↓

Worker Worker Worker

The dataset can be partitioned and processed across the cluster.

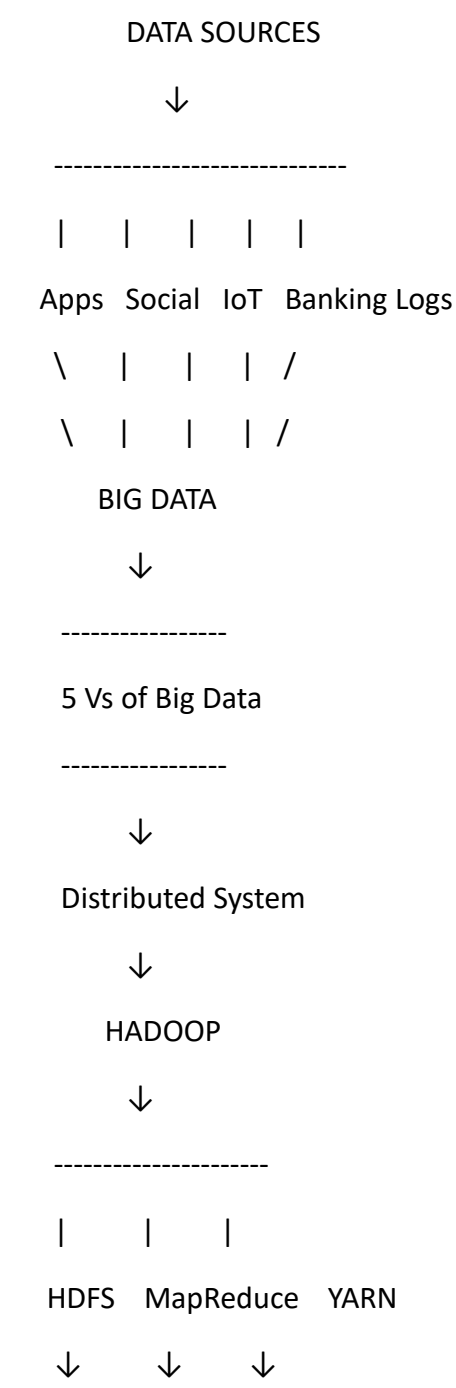
Dataset

Simply:

“A collection of data.”

RDD is an immutable, partitioned collection of data that Spark can process in parallel across a cluster.

The COMPLETE Big Data story



Storage Processing Resources

↓

Blocks

↓

NameNode + DataNodes

↓

Replication / Reliability

OR / AND

SPARK

↓

Distributed Processing

↓

Driver + Executors

↓

PySpark

↓

| |

RDD DataFrame

↓ ↓

Transformations Structured

+ Actions Analytics